

# Design Document: PostgreSQL semver Extension

## 1. Structure of the Extension:

An extension is not just one file. It is a group of files that each do a different job.

semver.c is the main file of the project. All the C code are here. Every function that PostgreSQL calls, whether it is reading a version string, comparing two versions, getting the major number, or computing a MAX aggregate, is written in this file. When we run make, this file gets compiled into semver.so, which is the plugin file that PostgreSQL loads into memory.

semver.control is a small metadata file. PostgreSQL reads this file first when someone types CREATE EXTENSION semver. It tells PostgreSQL the name of the extension, its version, and where to find the compiled library. Without this file, PostgreSQL cannot find our extension at all. semver--1.0.sql is the registration script. PostgreSQL does not automatically know about the C functions we wrote. We have to introduce them using SQL. This file runs automatically when CREATE EXTENSION semver is called. It tells PostgreSQL about the new type, the functions that read and display it, the operators, the aggregates, and the bonus cast functions. Without this file, our C code would compile fine but PostgreSQL would never know it exists.

Makefile handles the build process. We did not write a complicated build script. We used PostgreSQL's own build system called PGXS, which only needed four lines of settings from us. It knows where all the PostgreSQL headers are, what compiler flags to use, and where to copy the final files.

sql/semver\_test.sql contains all the regression test queries. It covers basic input and output, invalid input rejection, all six comparison operators, all five utility functions, ORDER BY, MAX and MIN aggregates, and a combined query using is\_compatible.

expected/semver\_test.out contains the exact expected output of every query in the test script. PostgreSQL's test runner compares the actual output against this file character by character. Any difference is treated as a failure.

demo.sql is a standalone script that loads 20 real software packages and demonstrates every feature of the extension. A grader can run it with psql mydb -f demo.sql and see everything working in one screen.

## 2. The Role of the Magic Block:

PostgreSQL is careful about which libraries it loads. Before loading any extension, it checks whether the library was compiled for the same version of PostgreSQL that is currently running. This check happens through something called the magic block:

```
c
```

```
PG_MODULE_MAGIC;
```

This one line goes at the top of semver.c. It puts version information into our compiled library. When PostgreSQL loads semver.so, it reads this information and checks it against its own version. If they match, the loading continues. If they do not match, for example if we compiled against PostgreSQL 15 but version 16 is running, PostgreSQL refuses to load the library and shows an error.

The rules are strict. It must appear in exactly one file. If we forget it, PostgreSQL will not load. If we put it in two files, the compiler gives a duplicate symbol error. We put it at the top of `semver.c` and did not touch it again.

### 3. The Role of PGXS :

When we compile normal C code, we have to tell the compiler where header files are, what flags to use, and where to put the output. PostgreSQL has many internal header files in many folders. Getting all the compiler settings right by hand would be very difficult.

PGXS solves this. It is a Makefile fragment that comes with PostgreSQL's development package. Our entire Makefile only has four variable settings:

makefile

MODULE\_big = semver

OBJS = semver.o

EXTENSION = semver

DATA = semver--1.0.sql

After that, we include PGXS and it handles everything. When we run `make`, PGXS finds the right include paths, the right compiler flags, and the right output format. When we run `sudo make install`, PGXS copies `semver.so` to PostgreSQL's library folder and copies `semver.control` and `semver--1.0.sql` to PostgreSQL's extension folder.

Four lines of settings and the whole build system works. The `pg_config` tool that PGXS uses internally knows everything about the current PostgreSQL installation on our machine.

### 4. The In-Memory Representation of the Type :

When PostgreSQL stores a `semver` value, either in a table or as a result during a query, it needs a place to put it in memory. We made a simple C struct for this:

c

```
typedef struct {  
    int32 major;  
    int32 minor;  
    int32 patch;  
} Semver;
```

Each field is an `int32`, which is a 32-bit integer. There are three fields and each takes 4 bytes, so the struct is exactly 12 bytes total. This is why we wrote `INTERNALLENGTH = 12` in the SQL type definition. We are telling PostgreSQL exactly how much space each `semver` value needs.

When someone inserts '16.4.0' into a `semver` column, this is what happens in memory:

| major = 16 (4 bytes) | minor = 4 (4 bytes) | patch = 0 (4 bytes) |

Total = 12 bytes

PostgreSQL saves these 12 bytes to disk. When it needs to show the value, it reads the 12 bytes back, rebuilds the struct, and calls our `semver_out` function which writes it back as "16.4.0".

## 5. The Comparison Helper :

The most important function in our extension is one that is never called directly from SQL. It is a private C helper called `semver_compare`:

```
c
static int semver_compare(Semver *a, Semver *b)
{
    if (a->major != b->major)
        return a->major - b->major;
    if (a->minor != b->minor)
        return a->minor - b->minor;
    return a->patch - b->patch;
}
```

This function is the main reason the extension exists. It compares two semver values field by field using integer math, not string comparison. The return value follows the standard C rule: negative means a is less than b, zero means they are equal, positive means a is greater than b. The important test case is 1.10.0 versus 1.9.0. String comparison gets this wrong because it compares character by character and thinks 1.10.0 comes before 1.9.0 alphabetically. Our function compares the minor fields as integers: 10 is greater than 9, so 1.10.0 is correctly seen as the larger version. Our regression test confirmed this with the query:

```
sql
SELECT '1.10.0'::semver > '1.9.0'::semver AS gt_critical;
-- returns: t
```

All six comparison operators call this one helper. The comparison logic lives in one place. If we ever need to fix a bug, we fix it once and all six operators get the fix automatically.

## 6. The SQL Registration Pattern :

Writing the C functions is only half the work. PostgreSQL also needs to know those functions exist, what inputs they take, and what they return. This is done in `semver--1.0.sql`.

Every C function we expose to SQL follows the same pattern. First in the C file:

```
c
PG_FUNCTION_INFO_V1(semver_lt);
Datum semver_lt(PG_FUNCTION_ARGS)
{
    Semver *a = (Semver *) PG_GETARG_POINTER(0);
    Semver *b = (Semver *) PG_GETARG_POINTER(1);
    PG_RETURN_BOOL(semver_compare(a, b) < 0);
}
```

Then in the SQL file:

```
sql
CREATE FUNCTION semver_lt(semver, semver)
    RETURNS bool
    AS 'MODULE_PATHNAME', 'semver_lt'
    LANGUAGE C IMMUTABLE STRICT;
```

MODULE\_PATHNAME gets replaced by the actual path to semver.so at runtime. IMMUTABLE tells PostgreSQL this function always gives the same result for the same inputs. STRICT means if either input is NULL, return NULL without calling our code.

For operators we add one more step:

sql

```
CREATE OPERATOR < (  
    LEFTARG    = semver,  
    RIGHTARG   = semver,  
    PROCEDURE  = semver_lt,  
    COMMUTATOR = >,  
    NEGATOR    = >=
```

```
);
```

COMMUTATOR tells the planner that a less than b is the same as b greater than a. NEGATOR tells it that NOT a less than b is the same as a greater than or equal to b. These hints help PostgreSQL optimize queries that use our operators.

We followed this same pattern for all six comparison operators, all five utility functions, and the aggregate functions. Once we understood it for one function, the rest were easy to write.

## 7. The Aggregate Functions :

Phase 3 required us to make MAX and MIN work on semver columns. PostgreSQL's aggregate system works by calling a state transition function on each row one at a time.

For MAX, the state transition function is semver\_larger. It takes the current running maximum and the next value and returns whichever is bigger:

c

```
PG_FUNCTION_INFO_V1(semver_larger);  
Datum semver_larger(PG_FUNCTION_ARGS)  
{  
    Semver *a = (Semver *) PG_GETARG_POINTER(0);  
    Semver *b = (Semver *) PG_GETARG_POINTER(1);  
    PG_RETURN_POINTER(semver_compare(a, b) >= 0 ? a : b);  
}
```

Then we register it as an aggregate:

sql

```
CREATE AGGREGATE max(semver) (  
    SFUNC = semver_larger,  
    STYPE = semver  
);
```

PostgreSQL handles all the looping. It feeds the first row as the initial state, then passes each subsequent row through semver\_larger. The final state is the result. MIN works the same way using semver\_smaller which returns the smaller of the two values.

We tested both and confirmed:

```
SELECT max(version) FROM packages; -- returns 24.0.7
```

```
SELECT min(version) FROM packages; -- returns 1.6.4
```

## 8. The Regression Test Strategy :

We created two files: `sql/semver_test.sql` with all the test queries and `expected/semver_test.out` with the exact expected output.

Our test script covers everything the document required:

Basic input and output tests confirm that valid version strings are parsed and displayed correctly, including edge cases like `0.0.0` and `20.10.0`.

Input rejection tests confirm that malformed strings like `'1.2'`, `'1.2.3.4'`, `'v1.2.3'`, `'1.2.x'`, `"` and `'-1.2.3'` all produce the correct error message with the hint we wrote.

Comparison operator tests include the critical case of `1.10.0` versus `1.9.0` which verifies that our comparison is integer-based and not string-based. We tested all six operators with cases for patch differences, minor differences, and major differences.

Utility function tests cover `major`, `minor`, `patch`, `bump_minor` with multiple inputs including edge cases like `bump_minor('1.9.3')` which must return `1.10.0`, and all four cases of `is_compatible`.

`ORDER BY` tests insert seven packages and verify they sort correctly in both ascending and descending order.

`MAX` and `MIN` tests confirm the aggregate functions return the correct highest and lowest versions.

Combined query runs `is_compatible` across the full test table, matching the demo from the project manual.

The workflow we used was to run the test script in `psql` first, copy the output, and save it as the expected file. Once `make installcheck` passed, we left the expected file alone.

## 9. Design Decision:

When we wrote the `semver_in` function, we had to decide how to read a string like `"1.2.3"` and turn it into our struct. There were two options.

Option 1 was to write a manual parser. We could loop through each character one by one, collect digits, check for dots, and build the numbers ourselves. This gives full control but requires writing around 30 to 40 lines of careful code with many edge cases to handle.

Option 2 was to use `sscanf` with a trick. This is what we chose:

```
c
n = sscanf(str, "%d.%d.%d%c",
           &result->major,
           &result->minor,
           &result->patch,
           &extra);
```

```
if (n != 3 || result->major < 0 || result->minor < 0 || result->patch < 0)
    ereport(ERROR, ...);
```

The interesting part is the `%c` at the end. `sscanf` returns how many items it successfully read. If someone passes `"1.2.3"`, `sscanf` reads three integers and stops, returning 3. If someone passes `"1.2.3.4"` or `"1.2.3abc"`, the `%c` catches the extra character and `sscanf` returns 4. We check that `n` is exactly 3, which means exactly three integers and nothing extra.

We also check that all three fields are not negative, which rejects inputs like `"-1.2.3"`.

The first reason we chose this approach is simplicity. Our sscanf approach does the same job in 8 lines that a manual parser would do in 40 lines. Less code means fewer chances for bugs. The second reason is that it already handles everything we need. Our regression tests confirmed that all of these are correctly rejected: "1.2", "1.2.3.4", "v1.2.3", "1.2.x", "" and "-1.2.3". The third reason is performance. sscanf is a well-tested standard library function optimized for decades. Writing our own parser would almost certainly be slower and less reliable. The only trade-off is that sscanf error messages are not very specific about which part of the string is wrong. But since our ereport message shows the full invalid string and our hint explains the correct format, this is perfectly acceptable.

## 10. Bonus Features :

### Bonus 10.1: B-tree Operator Class for Indexing

We registered a B-tree operator class for semver so that CREATE INDEX works on semver columns. This required assigning our seven comparison functions to specific strategy numbers inside an OPERATOR CLASS declaration:

```
sql
CREATE OPERATOR CLASS semver_ops
    DEFAULT FOR TYPE semver USING btree AS
        OPERATOR 1 <,
        OPERATOR 2 <=,
        OPERATOR 3 =,
        OPERATOR 4 >=,
        OPERATOR 5 >,
        FUNCTION 1 semver_cmp(semver, semver);
```

The strategy numbers 1 through 5 correspond to less than, less than or equal, equal, greater than or equal, and greater than. The support function semver\_cmp returns a negative, zero, or positive integer which is exactly what B-tree needs for sorting.

We verified this bonus by creating an index and running EXPLAIN with sequential scan disabled:

```
sql
CREATE INDEX idx_packages_version ON packages USING btree (version);
SET enable_seqscan = off;
EXPLAIN SELECT * FROM packages WHERE version >= '2.0.0'::semver;
```

The query plan showed an Index Scan instead of a Seq Scan, confirming the index is being used correctly.

### Bonus 10.3: Custom Cast Between text and semver

We registered explicit casts between text and semver so users can write text\_column::semver naturally. This required two new C functions and two SQL registrations.

The cast from text to semver:

```
c
PG_FUNCTION_INFO_V1(semver_from_text);
```

```
Datum semver_from_text(PG_FUNCTION_ARGS)
```

```
{  
    text *txt = PG_GETARG_TEXT_PP(0);  
    char *str = text_to_cstring(txt);  
    /* same parsing logic as semver_in */  
}
```

The cast from semver to text:

c

```
PG_FUNCTION_INFO_V1(semver_to_text);
```

```
Datum semver_to_text(PG_FUNCTION_ARGS)
```

```
{  
    Semver *v = (Semver *) PG_GETARG_POINTER(0);  
    char *str = palloc(32);  
    snprintf(str, 32, "%d.%d.%d", v->major, v->minor, v->patch);  
    PG_RETURN_TEXT_P(cstring_to_text(str));  
}
```

Registered in SQL as:

sql

```
CREATE CAST (text AS semver) WITH FUNCTION semver_from_text(text) AS IMPLICIT;
```

```
CREATE CAST (semver AS text) WITH FUNCTION semver_to_text(semver);
```

We verified this bonus works by testing:

sql

```
SELECT '1.2.3'::text::semver;      -- works via cast
```

```
SELECT '1.2.3'::semver::text;      -- works via cast
```

Both returned the correct values without any errors.